L10: Stored Procedures & Functions

Sirawich.V

Ref. https://www.mysqltutorial.org/

RECAP from Lab #9

- View

```
CREATE VIEW salePerOrder AS
    SELECT
        orderNumber,
        SUM(quantityOrdered * priceEach) total
    FROM
        orderDetails
    GROUP by orderNumber
    ORDER BY total DESC;
```

10.1 Delimiter

The default delimiter for a single MySQL statement is semicolon (;).
If we need to group many statements as one statement, it is necessary to temporarily redefine the delimiter so that you can pass the entire stored procedure to the server as a single statement.

Syntax

```
DELIMITER delimiter_character
```

The delimiter_character may consist of a single character or multiple characters, such as `//` or `$$`. However, you should avoid using the backslash (`\`) because it's the escape character in MySQL.

Delimiter (revert back to default ;)

Just set it back!!

Syntax

```
DELIMITER ;
```

10.2 What is Stored Procedure?

A stored procedure is a reusable set of instructions that perform a specific task.

Advantage

- easy to reuse and modify code
- empower SQL to accomplish the task with programming constructs.
- security by eliminating direct access to the table
- reduced network traffic (only the procedure name is passed)
- performance in client-server applications.

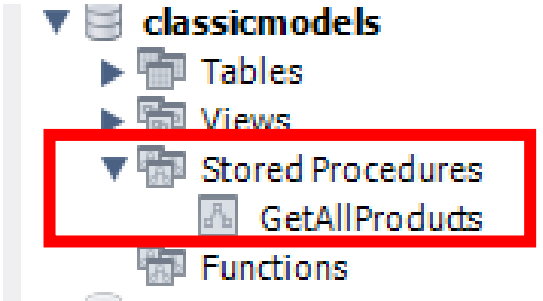
CREATE PROCEDURE statement

Syntax

```
DELIMITER //
CREATE PROCEDURE sp_name(parameter_list)
BEGIN
    statements;
END//
DELIMITER ;
```

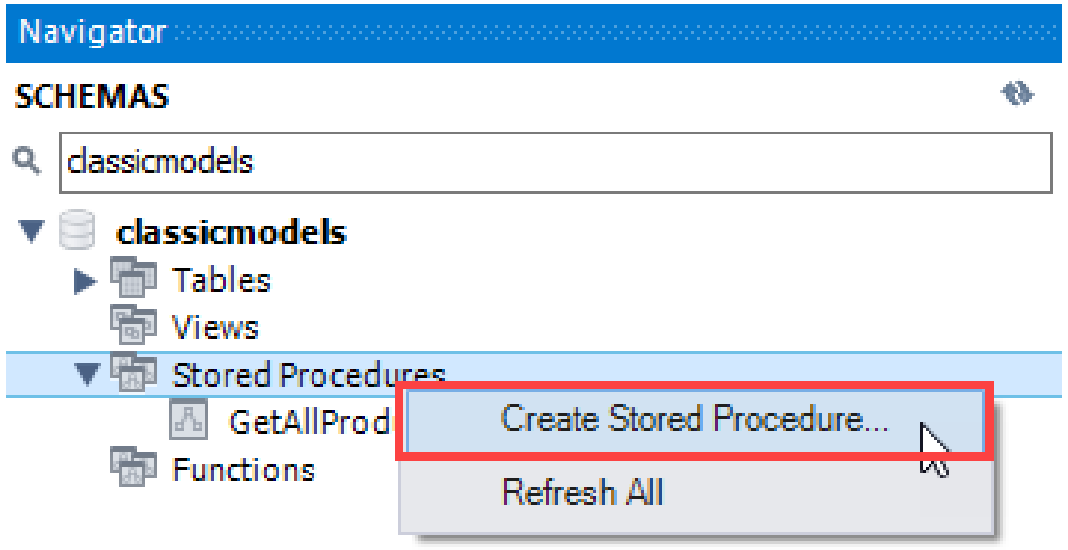
Example

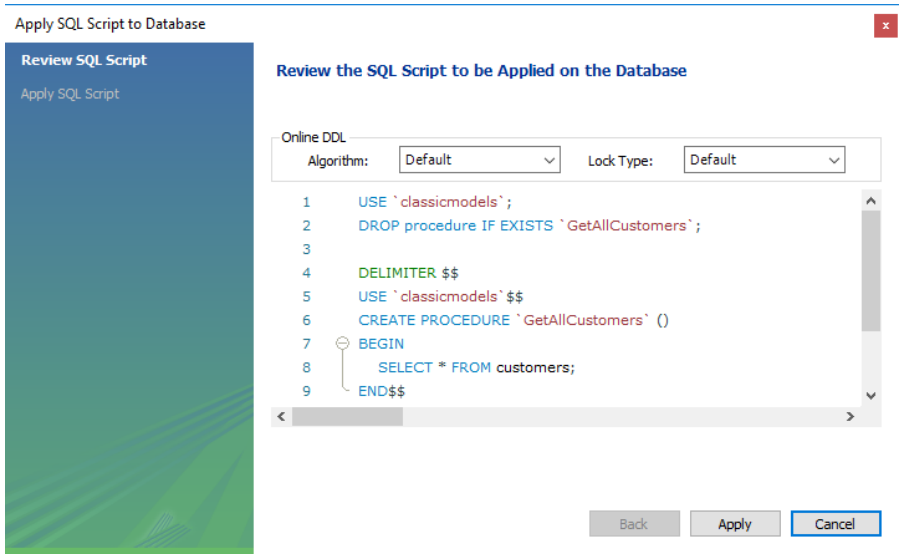
```
DELIMITER //
CREATE PROCEDURE GetAllProducts()
BEGIN
SELECT *  FROM products;
END //
DELIMITER ;
```



CREATE PROCEDURE statement

Create by Workbench wizard
https://www.mysqltutorial.org/mysql-stored-procedure/getting-started-with-mysql-stored-procedures/





Executing a stored procedure

Syntax

```
CALL sp_name(argument_list);
```

Example

```
CALL GetAllProducts();
```

productCode	productName	productLine	productScale	productVendor	productDescription
S10_1678	1969 Harley Davidson Ulti...	Motorcycles	1:10	Min Lin Diecast	This replica features work
S10_1949	1952 Alpine Renault 1300	Classic Cars	1:10	Classic Metal Creations	Turnable front wheels; st
S10_2016	1996 Moto Guzzi 1100i	Motorcycles	1:10	Highway 66 Mini Classics	Official Moto Guzzi logos a
S10_4698	2003 Harley-Davidson Eag...	Motorcycles	1:10	Red Start Diecast	Model features, official Ha
S10_4757	1972 Alfa Romeo GTA	Classic Cars	1:10	Motor City Art Classics	Features include: Turnab

Process the PROCEDURE

Alter Procedure:
https://www.mysqltutorial.org/mysql-stored-procedure/alter-stored-procedure/

Drop Procedure:
https://www.mysqltutorial.org/mysql-stored-procedure/mysql-drop-procedure/

10.3 Variables

A variable is a named data object whose value can change during the execution of a stored procedure. Typically, you use variables to hold immediate results.

- Declaring variables

  Syntax

```
DECLARE variable_name datatype(size);
```

- Assigning variables

  Syntax

```
SET @variable_name = value;
SELECT value INTO @variable_name;
```

Variables

Example

```
DELIMITER $$

CREATE PROCEDURE GetTotalOrder()
BEGIN
	DECLARE totalOrder INT DEFAULT 0;

	SELECT
		COUNT(*)
	INTO totalOrder FROM
		orders;

	SELECT totalOrder;
END$$

DELIMITER ;
```

totalOrder
326

User-defined variables

You can pass a value from an SQL statement to other SQL statements within the same session by storing the value in a user-defined variable.

Syntax

```
@variable_name
```

Note. User-defined variables are case-insensitive.

Parameters

A parameter in a stored procedure has three modes.

- **`IN`** is the default mode. The calling program must pass an argument to the stored procedure.
- The value of an **`OUT`** parameter can be modified within the stored procedure then passed back to the calling program.
- An **`INOUT`** parameter is a combination of IN and OUT parameters.

10.4 Defining a parameter

Syntax

```
IN parameter_name datatype;
```

Example: IN

```
DELIMITER //

CREATE PROCEDURE GetOfficeByCountry(
        IN countryName VARCHAR(255)
)
BEGIN
        SELECT *
        FROM offices
        WHERE country = countryName;
END //

DELIMITER ;
```

Example: IN

```
CALL GetOfficeByCountry('USA');
```

officeCode	city	phone	addressLine1	address
1	San Francisco	+1 650 219 4782	100 Market Street	Suite 30
2	Boston	+1 215 837 0825	1550 Court Place	Suite 10
3	NYC	+1 212 555 3000	523 East 53rd Street	apt. 5A

Defining a parameter

Syntax

```
OUT parameter_name datatype;
```

Example: OUT

```
DELIMITER $$

CREATE PROCEDURE GetOrderCountByStatus (
        OUT total INT
)
BEGIN
        SELECT COUNT(orderNumber)
        INTO total
        FROM orders
        WHERE status = 'Shipped';
END$$

DELIMITER ;
```

Example: OUT

```
CALL GetOrderCountByStatus(@total);
SELECT @total;
```

@total
303

Defining a parameter

Syntax

```
INOUT parameter_name datatype;
```

Example: INOUT

```
DELIMITER $$

CREATE PROCEDURE SetCounter(
       INOUT counter INT)
BEGIN
       SET counter = counter + 1;
END$$

DELIMITER ;
```

Example: INOUT

```
SET @counter = 1;
CALL SetCounter(@counter);
CALL SetCounter(@counter);
CALL SetCounter(@counter);
SELECT @counter;
```

@counter
4

10.5 What is Stored Function?

Stored Function is a reusable set of instructions that perform a specific task and return a single value.
(Same concept as programming function)

Advantage (same as stored procedure)

- easy to reuse and modify code
- empower SQL to accomplish the task with programming constructs.
- security be eliminating direct access to the table
- reduced network traffic (only the procedure name is passed)
- performance in client-server applications.

CREATE FUNCTION statement

Syntax

```
DELIMITER $$

CREATE FUNCTION function_name( param1, param2,...)
   RETURNS datatype
   [NOT] DETERMINISTIC
   BEGIN
      statements

   RETURN (variable);
   END $$
DELIMITER ;
```

CREATE FUNCTION statement

A deterministic function always returns the same result for the same input parameters, while a non-deterministic function produces different results for the same input parameters.

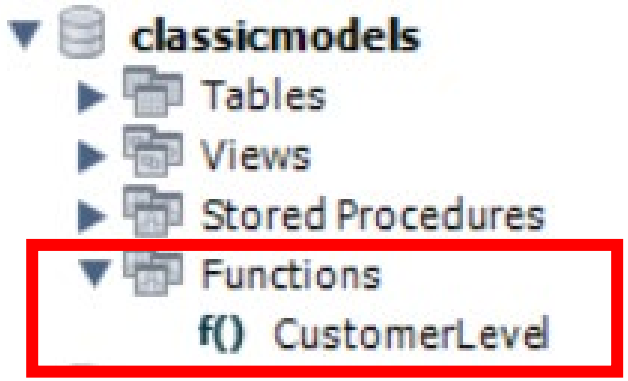
MySQL defaults to the `NOT DETERMINISTIC` option.

More about Deterministic:
https://www.geeksforgeeks.org/deterministic-and-nondeterministic-functions-in-sql-server/

CREATE FUNCTION statement

Example

```
DELIMITER $$
CREATE FUNCTION CustomerLevel(
        credit DECIMAL(10,2)
)
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
    DECLARE customerLevel VARCHAR(20);
    IF credit > 50000 THEN
                SET customerLevel = 'PLATINUM';
    ELSEIF (credit >= 50000 AND credit <= 10000) THEN
        SET customerLevel = 'GOLD';
    ELSEIF credit < 10000 THEN
        SET customerLevel = 'SILVER';
    END IF;
    RETURN (customerLevel);
END$$
DELIMITER ;
```



Executing a stored function

Syntax

```
SELECT sf_name(argument_list);
```

Example

```
SELECT
    customerName,
    CustomerLevel(creditLimit)
FROM
    customers
ORDER BY
    customerName;
```

customerName	CustomerLevel(creditLimit)
Alpha Cognac	PLATINUM
American Souvenirs Inc	SILVER
Amica Models & Co.	PLATINUM
ANG Resellers	SILVER
Anna's Decorations, Ltd	PLATINUM
Anton Designs, Ltd.	SILVER
Asian Shopping Network, Co	SILVER
Asian Treasures, Inc.	SILVER
Atelier graphique	GOLD
Australian Collectables, Ltd	PLATINUM
Australian Collectors, Co.	PLATINUM

Stored PROCEDURE vs FUNCTION

Feature	Stored Procedure (with OUT Parameter)	Function
Return Value	Uses OUT parameters to return values.	Returns a single value using RETURNS.
Usage in Queries	Cannot be used directly in SELECT statements.	Can be used inside SELECT, WHERE, JOIN, etc.
Multiple Outputs	Can return multiple values via OUT parameters.	Can return only a single value.
CALL Statement	Invoked using CALL procedure_name(...).	Invoked like SELECT function_name(...).
DML Operations (INSERT, UPDATE, DELETE)	Can modify database data.	Should not modify data (but MySQL allows it).
Performance	Slightly slower due to handling multiple parameters.	Faster for single-value computations.